ANALYSIS

HXC-119 (Feature/High) — ACP support investigation + implementation report

Status: DONE (Phase 1-4 adopted + verified; Phase 5 honestly deferred — operator design fork below)

STEP 1 — Investigation (FACT)

- `helix_code/internal/acp/` did **not** exist on this worktree's branch (`worktree-agent-a8de61cfd1f82404a`, based on outer-repo commit `6bf1be7a`). `git log --oneline --all -- helix_code/internal/acp` however showed two commits touching that path — `ca76e14b` and `edbd5a49` — neither an ancestor of this worktree's HEAD.
- Traced those commits to local branch `feature/helixllm-full-extension` (tip `68722674`, present locally with no network fetch needed — `origin/feature/helixllm-full-extension` also carries them). Its merge-base with this worktree's HEAD is exactly `6bf1be7a` — i.e. this worktree's HEAD is a strict ancestor of that branch. A parallel track had **already implemented HXC-119 through Phase 4** (real ACP handshake + real streaming turn-generation), bundled together with HXC-117 (verifier CONST-040 capability flags) and HXC-118 (RAG) in the same two commits. `docs/workable_items.db` on that branch carries HXC-119 as `Queued` (Feature/High) — source landed, DB-close deliberately deferred (a documented "F-DBTOOL" sync-tooling finding, unrelated to ACP itself).
- Per §11.4.74 (extend-don't-reimplement), §11.4.124 (investigate-before-duplicating/removing), and §11.4.181 (one feature ⇒ one canonical branch), reimplementing ACP from scratch here would have created a **second, divergent implementation of the same feature on a different branch** — exactly the failure class those anchors exist to prevent. Instead I adopted the real, already-tested implementation.

Established sibling-capability pattern (for CONST-040: MCP/LSP/ACP/…)

- MCP: `internal/mcp` (client package: transport_stdio/http/sse/ws, registry, lifecycle, oauth) + `cmd/cli/mcp_cmd.go` (dedicated cobra subcommand, dispatched in `main()` before `flag.Parse()`).
- LSP: `internal/tools` (LSPManager) + `cmd/cli/lsp_cmd.go`, same dispatcher shape, delegating to a shared `internal/commands.LSPCommand` renderer.
- ACP follows the SAME shape: `internal/acp` (the *agent*-side protocol package) + `cmd/cli/acp_cmd.go`, dispatched in `main()` identically to `mcp`/`lsp`. Confirmed correct: HelixCode is itself an agentic coding CLI (like Claude Code / Gemini CLI), so the correct ACP *role* for it is **Agent** (the side an ACP-aware editor such as Zed/JetBrains spawns as a subprocess and drives) — not Client. This matches `docs/research/07.2026/05_mcp_acp_protocols/` and `docs/research/const040_capability_model_20260712/DESIGN.md` already present on the feature branch (read for context, not modified — docs/ out of scope for this commit).

STEP 2 — Deep research (§11.4.150, ≥2 angles, cited, dated 2026-07-12)

1. WebFetch `https://agentclientprotocol.com/protocol/overview` — confirmed JSON-RPC 2.0 request/response + notification split, Agent vs Client role definitions, core method inventory (`initialize`, `session/new`, `session/prompt`, `session/cancel`, `session/update` notification, client-side `fs/*`/`terminal/*`/`session/request_permission`).
2. WebFetch `https://agentclientprotocol.com/protocol/initialization` — confirmed `protocolVersion` is an integer, `clientCapabilities`/ `agentCapabilities` shapes (fs.readTextFile/ writeTextFile, terminal, mcpCapabilities, promptCapabilities, loadSession, authMethods).
3. WebSearch “agentclientprotocol.com stdio transport newline-delimited JSON-RPC stdout stderr” → confirmed newline-delimited JSON-RPC over stdin/stdout, stderr reserved for logs, no LSP-style Content-Length framing.
4. WebFetch `https://agentclientprotocol.com/protocol/prompt-turn` — confirmed `session/prompt` params (sessionId + content blocks) / result (`stopReason` enum: end_turn/max_tokens/max_turn_requests/ refusal/ cancelled) and `session/update` notification variants (agent_message_chunk, tool_call, tool_call_update, plan, usage_update).

All four confirmed against `internal/acp/agent.go`'s actual implementation (verbatim from the adopted commits) — protocolVersion negotiation, content-block flattening (`Text`/`ResourceLink`), and the exact `StopReason` mapping (`StopReasonEndTurn`/`StopReasonMaxTokens`/`StopReasonRefusal`/`StopReasonCancelled`) all match the spec as fetched today. No correction needed to the adopted code; confirms it is not a bluff and not missing a bigger problem — it is a faithful, minimal-viable ACP Agent. The repo's own prior research (`docs/research/07.2026/05_mcp_acp_protocols/`, `docs/research/const040_capability_model_20260712/DESIGN.md`, both present on `feature/helixllm-full-extension`, read but not modified) independently reached the same conclusions.

STEP 3 — Implementation (adopted, scoped to HXC-119 only)

Extracted ONLY the ACP-scoped hunks from the bundled upstream commits (`ca76e14b`, `edbd5a49`), explicitly excluding the co-mingled HXC-117 (verifier `VerificationResult.Supports{MCP,LSP,ACP,RAG,Skills,Plugins}` fields + CLI capability-flag rendering) and HXC-118 (RAG retriever/ embedder) changes bundled in the same commits — those are out of scope for this ticket and depend on verifier/RAG code this worktree does not have.

Files added/changed (verbatim content from the source commits, wiring hand-verified line-by-line against this worktree's actual `main.go`): - `helix_code/internal/acp/doc.go` — package doc, scope boundary, constitutional anchors. - `helix_code/internal/acp/agent.go` — the `Agent` type: `Initialize`, `Authenticate` (honest `MethodNotFound`), `Logout` (honest no-op), `NewSession` (real UUID session tracking), `Prompt` (real `provider.GenerateStream` delegation + real `session/update` streaming + real `Cancel`-driven `context.CancelFunc`), `CloseSession`, `ListSessions`/`ResumeSession`/`SetSessionConfigOption`/`SetSessionMode` (honest `MethodNotFound` — not advertised as supported by `Initialize`). - `helix_code/internal/acp/agent_test.go` — 6 tests: real handshake, real session+prompt honest-error paths, real `GenerateStream` delegation proof, real truncated-stop-reason mapping, real in-flight cancellation. - `helix_code/cmd/cli/acp_cmd.go` — `helixcode acp` cobra command; wires `os.Stdin`/`os.Stdout` (or test doubles) to a real `acpsdk.NewAgentSideConnection`, resolves a real LLM provider via the pre-existing `buildSubagentLLMProvider`. - `helix_code/cmd/cli/acp_cmd_test.go` — 1 test: real `acpsdk.ClientSideConnection` ↔ `acpsdk.NewAgentSideConnection` handshake over a real `io.Pipe`, driven through the cobra `RunE`. - `helix_code/cmd/cli/main.go` — ONLY the `os.Args[1] == "acp"` dispatcher block (mirrors the existing `"mcp"` dispatcher immediately above it); none of the HXC-117 `formatCapabilityFlags`/RAG hunks from the same upstream diff were brought in. - `helix_code/cmd/cli/i18n/`

bundles/active.en.yaml — ONLY the 3 ACP keys (cli_acp_root_short, cli_acp_root_long, cli_acp_listening); the HXC-117 capability-flag i18n keys from the same upstream diff were excluded. - helix_code/go.mod / go.sum — ONLY the github.com/coder/acp-go-sdk v0.13.5 require + its two go.sum entries; the digital.vasic.rag (HXC-118) require was excluded.

The gap and how it was closed

Gap: no ACP package, no helixcode acp entrypoint, CONST-040's ACP capability had zero implementation anywhere reachable from this worktree. Closed by adopting the real, tested Phase 1-4 implementation described above: real JSON-RPC 2.0 handshake, real session tracking, real turn generation via the existing llm.Provider.GenerateStream (BLUFF-001-clean: no simulated text), real cancellation.

Genuine operator-design fork (NOT guessed at, explicitly deferred)

ACP permission-request mapping is HXC-119 Phase 5 — mapping ACP's session/request_permission client-callback onto HelixCode's existing internal/approval / internal/tools/permissions subsystems. This is explicitly flagged in the adopted doc.go as "medium-high risk, requires its own security-focused review pass" and was deliberately NOT implemented by the original author either. Prompt currently only streams model text (no tool-call/file-write action taken on the client's behalf), so it never needs to request permission in this phase — every ACP method that WOULD need permission-mapping returns an honest acp.NewMethodNotFound / acp.NewInternalError rather than a fabricated success.

The exact operator question for Phase 5 (2-4 concrete options — not guessed, not implemented without a decision): 1. **(A) Map 1:1 onto `internal/approval`'s existing rule/decision model** — reuses today's approval policy engine verbatim; ACP's PermissionOption choices become approval-rule outcomes. Fastest, but ACP's per-tool-call granularity may not fit internal/approval's existing shape without extension. 2. **(B) Map onto `internal/tools/permissions`'s tool-execution gate** instead — closer semantic fit (ACP permission requests gate a specific tool call, same as this subsystem), but requires wiring Agent.Prompt to actually dispatch HelixCode's own tool-execution path when the remote editor requests a file write/command run via ACP — a materially larger change than (A). 3. **(C) New minimal ACP-specific permission adapter** that neither reuses nor extends either subsystem — smallest, most isolated diff, but duplicates policy logic that (A)/(B) would have reused, and is the option §11.4.74 (extend-don't-reimplement) discourages by default. 4. **(D) Defer Phase 5 to a dedicated future ticket with its own security-focused review pass**, as already recorded on the

feature branch — the item stays honestly Queued/partial for ACP's tool-execution surface until that review lands. *(Recommended, matching the existing design's own stated plan and this subagent's time-box — Phase 5 genuinely requires its own security review, not a scoped subagent's unilateral call.)*

Verify (all captured, this session, this worktree)

```
$ go build -tags=nogui ./cmd/... ./internal/...        # exit 0, no output
$ go vet   -tags=nogui ./cmd/... ./internal/...        # exit 0, no output
$ go test  -tags=nogui -v -count=1 ./internal/acp/...
--- PASS: TestNewAgentSideConnection_WiresWithoutPanic (0.00s)
--- PASS: TestAgent_RealHandshake (0.00s)
--- PASS: TestAgent_NewSessionThenPromptHonestErrors (0.00s)
--- PASS: TestAgent_PromptDelegatesToRealGenerateStream (0.00s)
--- PASS: TestAgent_PromptTruncatedStopReason (0.00s)
--- PASS: TestAgent_CancelStopsRealInFlightGeneration (0.05s)
ok      dev.helix.code/internal/acp 0.090s
$ go test  -tags=nogui -v -count=1 -run ACP ./cmd/cli/...
--- PASS: TestNewACPCommand_RealHandshakeOverStdioTransport (0.00s)
ok      dev.helix.code/cmd/cli  0.082s
$ go test  -tags=nogui -count=1 ./cmd/cli/...           # full package, ze
ok      dev.helix.code/cmd/cli  9.872s
ok      dev.helix.code/cmd/cli/i18n 0.002s
$ grep -rniE "\bsimulated\b|\bfor now\b|TODO implement|in production this
    internal/acp cmd/cli/acp_cmd.go cmd/cli/acp_cmd_test.go | ...   # clea
```

Note: this worktree's `submodules/*` (dag_orchestrator, helix_agent, helix_qa, etc — 38 replace-target modules referenced from `go.mod`) were NOT initialized when this task began (pre-existing environment gap unrelated to HXC-119 — the initial `go build ./cmd/... ./internal/...` failed with `reading .../go.mod: no such file or directory` for unrelated packages before I touched anything, confirmed by stashing my `main.go` edit and re-running the same failing build). I initialized them locally via `git submodule update --init --reference <main-checkout-path> <path>` — no network fetch, objects were already present in the main checkout's `.git/modules/*` — so the build/vet/test evidence above is real and complete, not degraded by that gap.

Test tally

- `internal/acp`: 6/6 PASS (real handshake, real session+prompt honest errors, real GenerateStream delegation, real truncated-stop-reason mapping, real in-flight cancellation).
- `cmd/cli` ACP-scoped: 1/1 PASS (real stdio transport handshake through the cobra command).

- `cmd/cli` full package: PASS, zero regressions from this change.
- Anti-bluff smoke: clean.

Worktree / commit

- Branch: `worktree-agent-a8de61cfd1f82404a` (isolated worktree; NOT pushed, per instructions).
- Commit: `2f7daa14` — “feat(HXC-119): implement ACP (Agent Client Protocol) agent — adopted from feature/helixllm-full-extension Phase 1-4”.
- `git status --porcelain` (tracked scope) after commit: clean — only the 9 intended files (`internal/acp/{doc,agent,agent_test}.go`, `cmd/cli/acp_cmd{,_test}.go`, `cmd/cli/main.go`, `cmd/cli/i18n/bundles/active.en.yaml`, `go.mod`, `go.sum`) were staged and committed. `docs/` and `docs/workable_items.db` were NOT touched, as instructed.